Highlights

A Reproducible Edge-to-Cloud Infrastructure for Sustainable Orchestration of Embedded WebAssembly IoT Devices

Luca D'Agati, Giuseppe Tricomi, Fabio Orazio Mirto, Francesco Longo, Giovanni Merlino

- Kubernetes CRDs express intent for constrained non-node IoT devices.
- A TLS/CBOR gateway bridges cloud orchestration and embedded WASM execution.
- Reconciliation policies preserve evidence ownership across cloud, fog, and edge.
- A 100-trial campaign reports 100% transactional success (Wilson LB 96.3%) and 1286.0 ms mean end-to-end latency for a minimal WASM module.
- Steady-state application-layer heartbeat traffic is 1728 B/h per device, excluding TLS and lower-layer overhead; gateway mediation averages under 5 MiB RAM.

A Reproducible Edge-to-Cloud Infrastructure for Sustainable Orchestration of Embedded WebAssembly IoT Devices

Luca D'Agati[a,b,*], Giuseppe Tricomi[a,b], Fabio Orazio Mirto[a], Francesco Longo[a,b] and Giovanni Merlino[a,b]

[a]Department of Engineering, University of Messina, Messina, Italy
[b]CINI: National Interuniversity Consortium for Informatics, Rome, Italy

ARTICLE INFO

Keywords:
Cloud–Fog–Edge continuum
digital research infrastructures
sustainable orchestration
green experimentation
Internet of Things
WebAssembly
embedded systems
Kubernetes

ABSTRACT

Cloud–Fog–Edge infrastructures increasingly need to deploy and observe application logic across cloud services, gateways, and embedded IoT devices, but conventional cloud-native orchestration assumes Linux-class nodes with container runtimes, resident agents, and rich networking. This leaves a gap for constrained endpoints that must remain lightweight while requiring secure attachment, life-cycle accountability, and reproducible evidence for sustainable research infrastructures. This work addresses the problem of expressing Kubernetes deployment intent for devices that cannot operate as Kubernetes nodes, and translating that intent into verifiable embedded execution. We propose a Kubernetes-native framework that models devices, applications, and gateways through Custom Resource Definitions (CRDs) and uses a gateway-mediated control path to bind device identity, supervise liveness, dispatch WebAssembly (WASM) workloads, and report execution evidence to the control plane. The gateway acts as the semantic boundary between declarative cloud state and constrained device protocols, preserving device simplicity while maintaining status ownership in Kubernetes. The framework is evaluated on a K3s-based test bed with emulated embedded targets across a complete enrollment-to-deployment workflow. Across 100 repeated trials, enrollment, heartbeat supervision, deployment, and transactional completion all succeeded. Heartbeat traffic stays under 2 KB/h per device, gateway pod footprint remains in the single-digit MiB and single-digit millicores, and the evaluated device firmware fits within 400 KiB of flash with no kubelet or container runtime. For sustainable research infrastructures, the framework provides a reproducible, measurement-ready substrate for energy-aware placement, instrumentation, and green experimentation policies.

1. Introduction

Digital research infrastructures (RIs), cyber-physical systems, and Internet of Things (IoT) deployments increasingly require software-defined behavior at the edge, where constrained devices execute application logic under reliability, security, timing, life-cycle, and sustainability constraints. Remote deployments can include sensors, actuators, lightweight gateways, and mixed-criticality nodes that must remain inexpensive and portable while still being supervised continuously. In edge-to-cloud RIs, this supervision is not only an operational requirement but also a prerequisite for reproducible experimentation and future energy or carbon accounting across heterogeneous sites. Traditional embedded deployment practices, including manual flashing, board-specific provisioning, and tightly coupled update channels, are workable at small scale but do not provide fleet-wide orchestration or reliable visibility into which software version is running on each device.

In parallel, cloud-native ecosystems have converged around declarative orchestration, reconciliation loops, immutable artifacts, and service-level observability. Kubernetes has become the de facto orchestration substrate for distributed services because it separates desired state from observed state and continuously drives the latter toward the former through controllers [46]. This model has transformed data-center operations, but it also exposes a gap when applied to embedded endpoints. Modern orchestrators assume Linux-capable nodes able to run containers, cluster agents, and rich networking stacks, whereas microcontrollers, sensors, and actuators generally cannot host these components. Nevertheless, cyber-physical deployments still need comparable guarantees for secure attachment, observability, and application life-cycle control.

WebAssembly (WASM) provides a practical execution abstraction for this gap. It supports portable, sandboxed workloads that can run across cloud, fog, and edge environments with a more stable module format than architecture-specific binaries [23]. For embedded systems, WebAssembly Micro Runtime (WAMR) and Zephyr-compatible integrations provide a concrete path toward microcontroller-oriented execution [11, 47]. Runtime feasibility alone, however, does not solve orchestration: a cloud-side intent must still be translated into a secure command for a constrained device, and execution evidence must be reflected back into the control plane.

This paper introduces a full-stack framework that addresses this problem through a gateway-centric architecture for the Cloud–Fog–Edge continuum. The framework builds an end-to-end path from Kubernetes CRDs on K3s, through a Rust/Transport Layer Security (TLS) gateway that mediates identity validation and state propagation, to Zephyr firmware running WAMR on embedded-class targets. In this path, the cloud records what should happen, the gateway

*Corresponding author.
Email address: ldagati@unime.it (L. D'Agati)
ORCID(s):

translates and validates that intent, and the device executes the WASM workload while reporting heartbeat-driven liveness and runtime outcomes. Embedded nodes therefore remain lightweight while still participating in a declarative orchestration model. For research infrastructures, the same path provides stable observation and actuation points that can be reused by sustainability dashboards, energy-aware schedulers, or green experiment-design workflows without moving cloud-native complexity into device firmware.

The framework is not presented as an energy-optimization algorithm, but as a reusable digital-research-infrastructure substrate that makes constrained edge participation observable, repeatable, and ready to be combined with future power, carbon-intensity, and placement policies.

The work is motivated by a concrete engineering observation: secure enrollment is necessary but insufficient. A practical platform must also define ownership of status updates, avoid false disconnection semantics during transient conditions, and expose enough evidence for operators to trust the control plane. This requirement defines the central novelty of the work: a Kubernetes-native evidence and reconciliation model for non-node embedded WASM endpoints, combining declarative resource modeling, secure gateway mediation, state reconciliation, and deployment reproducibility into a single verifiable operational path.

1.1. Scope and Contributions

This work presents a systems-oriented contribution at the intersection of IoT orchestration, embedded computing, and secure cyber-physical infrastructure. It does not claim novelty in WebAssembly, Kubernetes, TLS, or Concise Binary Object Representation (CBOR) as isolated technologies. Instead, it integrates these elements into a coherent platform that closes an implementation gap frequently acknowledged in the literature but less often documented end to end: how declarative cloud-side intent can be made operational on constrained, non-node embedded devices without sacrificing secure attachment, control-plane visibility, and state fidelity.

The contributions are fourfold:

- a Kubernetes-native resource model with CRDs for devices, gateways, and deployable WASM applications (Section 4);
- a gateway-mediated TLS/CBOR enrollment and control workflow with explicit identity matching and southbound message translation (Section 5);
- a reconciliation strategy that links desired and reported state while avoiding false disconnect loops and ambiguous status ownership (Section 5);
- an end-to-end K3s validation study with emulated embedded targets and sustainability-oriented observables for wire traffic, firmware footprint, and control-plane resource use, intended as baselines for later green-experimentation studies (Section 7).

The contribution of our work is to present as an enabling layer for sustainable digital RIs: it does not yet optimize energy use, but it makes edge-device participation declarative, repeatable, and instrumentable for subsequent power, carbon, and placement studies.

The remainder of the paper is organized as follows. Section 2 reviews related work. Section 3 formulates the problem and design requirements; Section 4 presents the architecture and resource model. Section 5 describes the southbound protocol and reconciliation policy; Section 6 summarizes the implementation stack. Section 7 reports the experimental evaluation. Section 8 discusses implications, validity, and reproducibility. Section 9 concludes the paper.

2. Related Work

This section reviews prior work along the axes that motivate the design in Sections 3–6: edge orchestration, constrained execution, secure IoT communication, reconciliation semantics, and reproducible evaluation. The main gap across these areas is the lack of an end-to-end path from Kubernetes-style declarative intent to verifiable execution on constrained embedded devices that cannot act as ordinary cluster nodes.

2.1. Cloud-Native Edge and IoT Orchestration

Foundational work on fog and edge computing established the need to move computation, storage, and control closer to data sources in order to reduce latency, improve locality, and support cyber-physical applications [7, 13, 39, 43]. Kubernetes has become the dominant orchestration substrate for cloud-native services because it provides declarative resource management, controllers, reconciliation loops, and extensibility through Custom Resource Definitions (CRDs) and operators [27, 46]. Lightweight distributions such as K3s and MicroK8s reduce deployment overhead and make cluster-style operation feasible at smaller sites [12, 35].

Several systems extend Kubernetes semantics toward the edge. KubeEdge moves parts of Kubernetes control closer to edge nodes, OpenYurt targets cloud-edge application management, Akri discovers and exposes leaf devices to Kubernetes, and Krustlet explored WebAssembly workloads as Kubernetes-scheduled units [2, 25, 26, 33]. Recent Kubernetes-native IoT orchestration based on Stack4Things (S4T), CRDs, and Crossplane abstractions similarly demonstrates the value of declarative resource management for IoT deployments [16]. These approaches motivate the present work, but they generally assume Linux-class edge hosts, gateway nodes, container-compatible execution environments, device plugins, or platform services attached to capable nodes. The target considered here is narrower and more constrained: an embedded endpoint that cannot run a kubelet, container runtime, Kubernetes agent, or full cloud-native network stack.

IoT middleware and industrial edge platforms provide additional context. EdgeX Foundry, FIWARE, oneM2M, OMA Lightweight M2M, and OPC UA address aspects of device modeling, interoperability, telemetry, and industrial communication [18, 22, 30–32]. These frameworks

are broader than a single orchestration mechanism and often emphasize interoperability or application enablement rather than Kubernetes-native desired-state reconciliation. The contribution is therefore not another general IoT middleware layer, but a focused integration that links cloud-style intent, gateway-mediated secure control, and embedded WASM execution through explicit resource ownership.

2.2. Constrained Execution and Secure Communication

Embedded and IoT operating systems provide the substrate on which constrained devices communicate and execute application logic. Zephyr, FreeRTOS, RIOT, and Contiki-NG represent different points in the design space for real-time behavior, connectivity, portability, and resource footprint [3, 5, 17, 47]; TinyOS established the long-standing importance of event-driven and resource-aware design in sensor-network systems [28]. These operating systems make embedded applications feasible, but they do not by themselves provide a cloud-side mechanism for expressing intent, binding identity, reconciling state, and verifying deployment outcomes.

WebAssembly adds a portable execution layer that can complement embedded operating systems. The original WebAssembly design emphasized safe, compact, and efficient portable code [23, 51], while the WebAssembly System Interface (WASI) extended the ecosystem toward non-browser execution [10]. Runtime implementations such as WAMR, Wasmtime, WasmEdge, and wasm3 demonstrate that WASM can execute outside browsers and, in some cases, in resource-constrained settings [9, 11, 49, 50]. Container and cloud ecosystems have also begun to integrate WASM through projects such as runwasi and Spin [15, 21]. These efforts establish runtime feasibility, but not operational orchestration: the unresolved question is how a cloud control plane can safely decide that a device should run a given module, transmit that intent through a trusted gateway, and receive evidence that execution and liveness match the requested state.

Secure IoT communication relies on standardized transport, object security, and compact serialization mechanisms. TLS 1.3 and DTLS 1.3 provide channel security [36, 37]; CoAP and MQTT address lightweight application-layer communication [29, 42]; OSCORE and COSE provide object security and compact cryptographic containers [40, 41]; and CBOR supports low-overhead binary serialization [8]. These mechanisms are complementary to the present work. A secure channel can protect messages in transit, but it does not determine which component owns reported state, when a device should be considered connected, or how a cloud resource should converge after a deployment acknowledgment.

2.3. Reconciliation, Evaluation, and Positioning

The operator pattern and Kubernetes controller model encode desired state, observe actual state, and repeatedly reconcile the two [27, 46]. In conventional clusters, this model works because nodes and workloads participate directly in the Kubernetes ecosystem. Edge systems complicate the model because connectivity can be intermittent, observations may be delayed, and devices may expose limited introspection. Prior platforms address parts of this challenge through edge agents, device twins, or local gateways [2, 18, 26, 32, 33]. For embedded targets, however, the gateway becomes more than a message forwarder: it is the semantic boundary between cloud resources and protocol-derived evidence.

Reproducible evaluation is equally important because physical hardware availability, network conditions, and manual setup can make embedded IoT experiments difficult to repeat. Large-scale experimental infrastructures such as SLICES highlight the role of programmable, shared scientific instruments for networking research [20]. Renode supports hardware-faithful emulation and multi-node embedded scenarios, while Cooja, ns-3, and OMNeT++ support repeatable simulation and network experimentation in wireless, sensor, and IoT research [4, 34, 38, 48]. The evaluation in this paper follows that direction by using an emulation-driven validation path and structured measurement records. This is appropriate because the main claim concerns end-to-end orchestration coherence rather than radio performance, energy consumption, or microarchitectural timing.

Taken together, existing cloud-edge orchestration systems extend Kubernetes toward capable edge nodes; IoT middleware improves interoperability and telemetry; embedded operating systems and WASM runtimes make constrained execution feasible; and secure communication standards provide protected message exchange. The proposed framework connects these strands at a narrower integration point: a Kubernetes-native control plane expressing declarative deployment intent for an embedded-class endpoint that remains outside the cluster and reports execution evidence through a gateway. Unlike middleware-centric approaches, the main abstraction is not a device model or telemetry bus, but the convergence between CRD intent, gateway evidence, and embedded WASM execution. The contribution is therefore best understood as a systems-integration study that combines explicit identity binding, gateway-mediated control, heartbeat-based liveness, CRD status convergence, and reproducible quantitative validation.

Table 1 summarizes frameworks' characteristics. The comparison is qualitative because the surveyed systems target different layers of the edge continuum, but it clarifies the specific gap addressed here: preserving Kubernetes-native intent and reconciliation while keeping the embedded endpoint outside the cluster and still obtaining execution evidence.

3. Problem Formulation and Design Requirements

The platform addresses the following problem:

> *How can a Kubernetes-based control plane reliably express the intent that a constrained device*

Table 1
Qualitative comparison relative to representative edge, IoT, and WASM orchestration approaches.

Approach	Primary abstraction	Assumed edge capability	Kubernetes integration	Gap for this work
KubeEdge/OpenYurt	Cloud-edge node and application management	Capable edge hosts or edge nodes with local agents	Extends Kubernetes control toward the edge	Too heavy for endpoints that cannot host cluster agents.
Akri/device-plugin style integration	Discovery and exposure of leaf devices	Kubernetes nodes near the device	Represents devices through cluster-attached resources	Does not make the constrained device itself a verifiable WASM execution target.
Krustlet/runwasi-style WASM execution	WASM workload as schedulable unit	Host able to participate in Kubernetes scheduling	Integrates WASM into Kubernetes workload execution	Targets worker-like execution environments rather than non-node embedded firmware.
IoT middleware	Device model, telemetry, interoperability, and management	Gateway, server, or device-management infrastructure	May be bridged to cloud platforms but is not primarily a CRD reconciliation model	Provides broad IoT management, but not Kubernetes-native deployment intent to embedded WASM execution evidence.
This framework	CRD intent plus gateway-mediated evidence	Zephyr/WAMR endpoint with TLS/CBOR protocol support only	Uses CRDs, controllers, and status reconciliation without making the device a worker node	Focused on reproducible orchestration of constrained embedded WASM targets.

should execute a specific WebAssembly module, route that intent through a secure gateway, and verify the resulting execution and liveness state, with measurable control-plane and southbound overhead compatible with sustainable research-infrastructure operation, without requiring the device to behave like a Kubernetes node?

Cloud orchestrators normally assume Linux-capable workers with container runtimes, cluster agents, and rich networking support. Embedded devices instead provide limited compute, memory, and network functionality, but they still need comparable guarantees for identity, observability, and life-cycle accountability. The framework therefore treats orchestration as an end-to-end translation problem between cloud intent, gateway-mediated protocol control, and edge-side WASM execution.

3.1. Design Pillars

The problem management is structured on three pillars, standing on the three level in which framework acts:

- At the resource-model level, Kubernetes assumes a declarative control plane with explicit status ownership, whereas embedded deployments often rely on imperative, device-local procedures that are opaque to the cloud.
- At the transport level, constrained devices need lightweight and secure communication paths, but the cloud side requires rich enough semantics to express enrollment, liveness, and application control.
- Operationally, even when protocol messages are correct, reconciliation logic must interpret transient conditions conservatively to avoid reporting an incorrect device state.

3.2. Design Requirements

These pillars lead to a compact set of requirements. Desired and observed state for devices, applications, and gateways must remain represented in Kubernetes resources. Enrollment must occur over a protected channel with auditable binding between transport-level identity and platform-level resource identity. The application life cycle must use a format usable across cloud, fog, and constrained edge targets, motivating WebAssembly as a common execution abstraction. Runtime evidence must confirm enrollment, heartbeat-based liveness, deployment status, and gateway association without manual inspection of device internals. For digital research infrastructures, the same evidence model should expose where workload placement, liveness, and execution outcomes can later be correlated with site-level power or carbon signals. Finally, the complete stack must deploy and test in a commodity research environment.

Table 2 makes these requirements explicit and links each one to the architectural decision and evaluation observable used later in the paper. This traceability is important because the system is not evaluated as a generic IoT middleware platform, but as an orchestration path whose correctness depends on agreement between cloud resources, gateway evidence, and device-side execution. A requirement is therefore considered satisfied only when the corresponding design element leaves an observable artifact in the control plane or in the measurement record.

Secure attachment without unified state would remain difficult to operate; unified state without secure attachment

Table 2
Traceability between design requirements, architectural decisions, and evaluation observables.

Requirement	Rationale	Design consequence	Evaluation observable
R1: Declarative intent	Operators need a durable statement of what should run even when a device is offline or temporarily unreachable.	Device, Application, and Gateway state is represented through Kubernetes CRDs rather than through ephemeral gateway memory alone.	Application desired state, Device phase, Gateway association, and status convergence.
R2: Identity binding	A protected channel is insufficient unless the endpoint is bound to the expected platform identity.	The gateway validates enrollment material before associating a TLS session with a Device resource.	Enrollment success, expected identity match, and gateway runtime view.
R3: Lightweight edge execution	Constrained endpoints cannot host kubelet, container runtimes, or Linux userspace services.	The device firmware executes portable WASM modules through WAMR on Zephyr.	Firmware footprint, WASM deployment result, and absence of edge cluster agents.
R4: Evidence-driven status	Control-plane state must describe observed execution rather than only command dispatch.	Gateway-propagated acknowledgments and results are treated as status evidence.	Deployment phase, execution outcome, heartbeat freshness, and transactional success count.
R5: Conservative liveness	Intermittent visibility must not immediately erase the latest valid attachment state.	Controllers avoid interpreting a single stale or incomplete observation as definitive disconnection.	Heartbeat observability, connectivity state, and cross-layer evidence checks.
R6: Reproducible sustainability baseline	Later energy and carbon policies require a repeatable orchestration substrate with known overhead.	The testbed records wire size, firmware footprint, and pod-level resource samples.	Bytes per hour per device, firmware flash/RAM footprint, gateway RAM, and gateway CPU.

would be insecure by construction; and either property without runtime evidence would leave operators unable to trust the control plane. Communication, orchestration, and status management are therefore co-designed rather than layered opportunistically. Embedded devices are not trusted to manage cluster state directly, and Kubernetes is not expected to maintain device sessions directly: a managed gateway acts as the trusted fog-layer mediation point while the control plane remains authoritative for desired state.

4. Architecture

4.1. Layered View

The framework is organized around the Cloud–Fog–Edge continuum. The **cloud layer** hosts the API server, dashboard, controllers, CRDs, and supporting services on Kubernetes; it stores desired state and persists reported execution state. The **fog layer** hosts the Rust-based gateway and emulation support services; it translates orchestration events into southbound TLS/CBOR messages, validates device identity, and propagates status updates back to the cloud. The **edge layer** contains Zephyr firmware, the WAMR execution environment, and deployed WASM modules; it executes the intended behavior and emits heartbeat and runtime evidence.

This decomposition localizes complexity where it is most affordable: Kubernetes retains declarative control, the gateway absorbs protocol and identity mediation, and the device remains lightweight. Figure 1 summarizes this path: declarative intent flows from the cloud toward the edge, while runtime evidence returns through the gateway into Kubernetes status fields.

4.2. Gateway-Centric Control

Gateway-centric mediation is the main architectural choice. Devices do not manipulate Kubernetes resources or embed cluster-specific logic. Instead, the gateway terminates southbound TLS sessions, validates enrollment messages, binds transport sessions to device identities, dispatches application life-cycle commands, and reports execution and liveness evidence to the cloud side.

This design keeps constrained devices small while concentrating protocol policy, identity checks, message normalization, and status propagation at a controllable boundary. It also gives the cloud side a stable integration surface if the device protocol evolves. Gateway correctness and availability are therefore explicit managed properties of the fog layer, rather than implicit assumptions distributed across constrained endpoints.

4.3. Resource Model

The framework relies on three primary CRDs, summarized in Table 3. The `Device` resource stores identity, gateway preference, runtime attributes, expected public key, and reported connectivity state. The `Application` resource expresses the desired WASM deployment for a target device and records execution outcomes in status fields. The `Gateway` resource describes endpoint metadata, capabilities, and the information needed to resolve the mediation path.

The resource model separates desired state from reported state. This preserves deployment intent when devices are offline, unreachable, or in transition, and it helps operators compare requested state with the latest evidence observed by the gateway and controllers.

4.4. Control and Observability Boundaries

The architecture distinguishes control ownership from evidence ownership. Operators and higher-level services

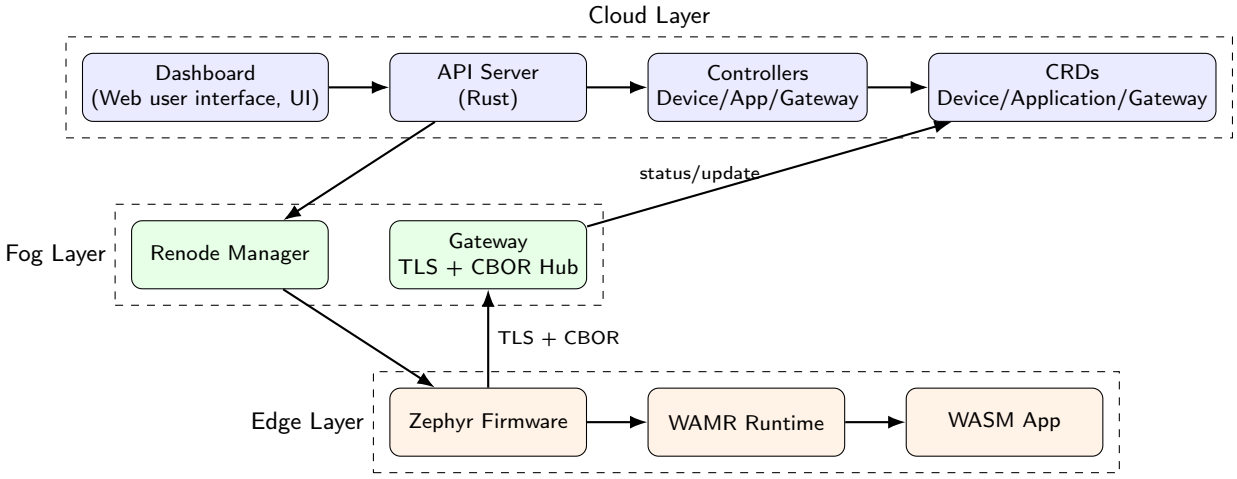

Figure 1: High-level architecture of the proposed framework. Declarative cloud intent is mediated by the gateway and translated into verifiable embedded WASM execution.

Table 3
Primary CRDs and control/evidence ownership.

Resource	Desired or configured state	Reported evidence
`Device`	Platform identity, expected key, gateway preference, runtime attributes	Gateway association, phase, connectivity, heartbeat freshness
`Application`	Target device, WASM artifact, desired execution state	Deployment phase, start outcome, execution result or error evidence
`Gateway`	Endpoint metadata, capabilities, namespace-scoped mediation path	Availability metadata, resolved devices, status propagation context

change desired state through Kubernetes resources; the gateway turns protocol-level observations into platform-level status updates; and controllers enforce reconciliation and consistency. This separation reduces ambiguous responsibility and limits conflicting status writes. The validation campaign specifically exercises this boundary by checking that transient visibility gaps do not overwrite fresh gateway evidence or produce false disconnection state.

5. Protocol and Reconciliation

5.1. Southbound Life-Cycle

The southbound protocol uses framed CBOR messages over TLS. It is compact enough for embedded-class clients, but still supports identity binding, acknowledgments, liveness evidence, and application deployment. The life-cycle has three phases: enrollment, heartbeat-based supervision, and resource-driven WASM deployment.

Enrollment establishes the transition from a protected transport session to a platform-recognized device identity:

1. The device opens a protected session with the gateway and requests enrollment.
2. The gateway validates the request before proceeding.
3. The device provides identity material that can be bound to a known platform resource.
4. The gateway compares that material with the identity registered in the control plane.
5. The device confirms that enrollment information has been received and that it is ready for managed operation.
6. The gateway stores the device–session association and updates the corresponding resource status.

After enrollment, periodic heartbeats maintain liveness visibility. At the communication layer, they show that the protected channel and device remain responsive. At the control-plane layer, they provide evidence used to refresh reported connectivity and device phase. Missing or delayed observations are interpreted conservatively, so transient gaps do not immediately become disconnection events.

WASM deployment follows the same evidence-driven pattern, but starts from declarative intent rather than from device attachment:

1. An operator or higher-level service declares that a target device should run a specific WASM workload.
2. The control plane observes the desired state and identifies the mediation path through the appropriate gateway.
3. The gateway translates the deployment intent into compact protected messages for the device.
4. The device firmware uses its embedded WASM runtime to load and start the workload.
5. The device returns execution evidence or an error indication to the gateway.

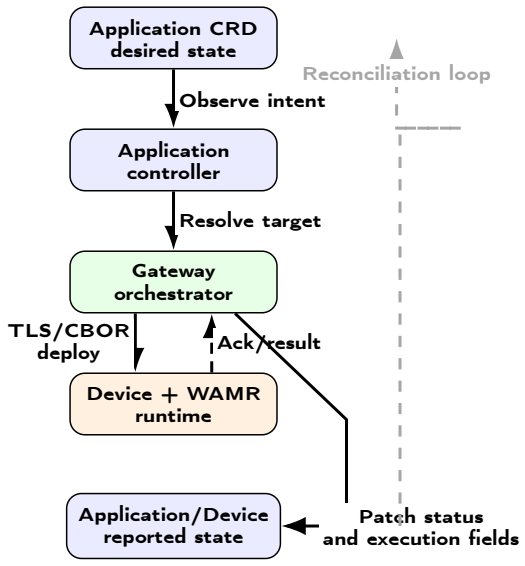

Figure 2: Application deployment workflow and status-reconciliation loop.

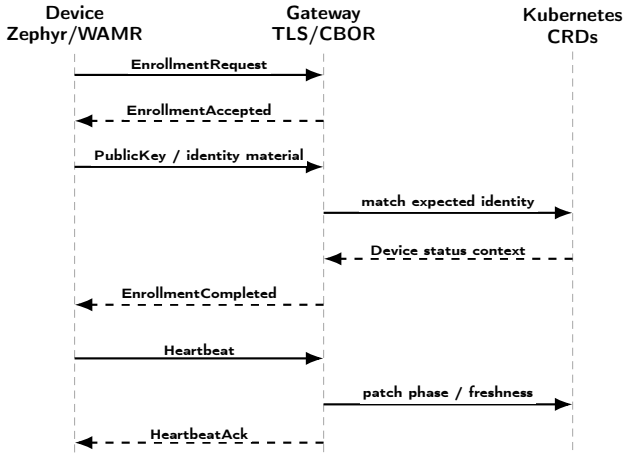

Figure 3: Enrollment and heartbeat sequence. Device-originated evidence is validated by the gateway and reflected into Kubernetes status.

6. The gateway propagates the reported outcome back into the control-plane status visible to operators.

Figure 2 summarizes the deployment loop: the controller resolves intent, the gateway mediates the southbound command, the device reports an outcome, and the control plane reconciles status. This preserves the declarative pattern and remains compatible with retry, staged rollout, and multi-gateway scheduling policies.

5.2. Protocol Sequences

Figures 3 and 4 summarize the southbound protocol interactions evaluated in Section 7. The diagrams emphasize role ownership: the device initiates attachment and emits runtime evidence, the gateway validates and translates protocol events, and the Kubernetes control plane stores desired and reported state.

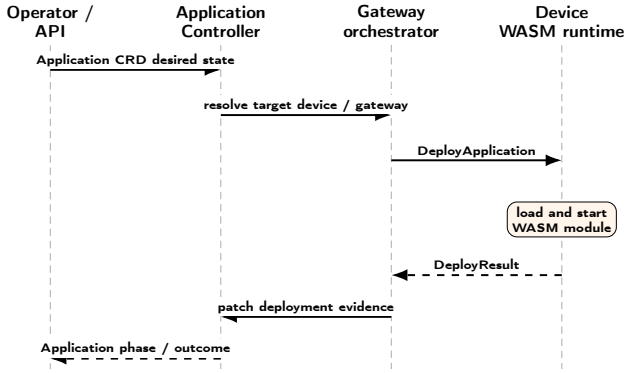

Figure 4: WASM deployment sequence. Declarative application intent is translated by the gateway into device execution and reconciled as application status.

During enrollment, the decisive event is not the opening of a protected transport session alone, but the successful association between the device-provided identity material and the `Device` resource expected by the control plane. Heartbeats then refresh the same association by producing liveness evidence that the gateway can interpret before patching Kubernetes status. Deployment follows the complementary direction: the `Application` resource records desired state, the controller resolves the mediation path, and the gateway converts intent into a southbound command. The returned `DeployResult` is treated as execution evidence, not merely as a transport acknowledgment, only after this evidence is propagated does application status converge to the reported outcome.

5.3. Reconciliation Policy

Protocol correctness becomes platform correctness only when message outcomes are interpreted consistently by the control plane. The framework therefore gives the gateway authority over protocol-derived observations and constrains controllers to reconcile those observations without overreacting to transient visibility gaps.

Integration testing exercised short-lived differences between gateway observations and control-plane status, confirming that liveness should not be inferred from a single incomplete observation. The resulting policy preserves the latest valid gateway association, avoids immediate disconnection when heartbeat evidence and transport visibility briefly diverge, and waits for a subsequent reconciliation cycle before treating the condition as a real state transition. Reconciliation policy is therefore part of the architecture rather than an incidental implementation detail.

For reproducibility, the protocol messages evaluated here should therefore be read as typed CBOR records with explicit identity, freshness, target, artifact, and outcome fields, even when the diagrams show only message names. The implementation-facing invariant is that a transport acknowledgment alone is insufficient for success: Kubernetes status is updated only after the gateway has matched the device identity, preserved the gateway

Table 4
Additive implementation-facing conventions for the validated gateway-mediated control path.

Element	Minimum evidence carried or recorded	Status/reconciliation use
Enrollment	Device identifier, firmware/runtime descriptor, device public-key material or fingerprint, gateway identifier, and freshness context for the protected session	The gateway binds the transport session to the expected `Device` resource before reporting connectivity or phase updates.
Heartbeat	Device identifier, gateway/session association, receive timestamp, and heartbeat freshness window	Controllers interpret heartbeat evidence conservatively: stale or missing observations do not overwrite the latest valid association until a subsequent reconciliation cycle confirms a state transition.
Deployment	Application identifier, target device, WASM artifact reference or payload hash, desired execution state, result status, and error or exit evidence when available	`Application` status converges only after gateway-propagated execution evidence; desired state remains intact when deployment evidence is missing, stale, or inconsistent.
Measurement record	Trial identifier, enrollment result, heartbeat-freshness check, deployment result, gateway runtime view, `Device` status, `Application` status, timeout outcome, and latency timestamps	A trial is counted as transactionally successful only when all evidence views agree; otherwise the trial is recorded as failed rather than inferred from a single eventually consistent field.

association, and propagated fresh deployment or liveness evidence. In the evaluated configuration, heartbeat freshness is interpreted relative to the configured 25 s firmware heartbeat period: a newly observed heartbeat refreshes the gateway association, while stale or missing observations are treated conservatively and do not immediately erase the last valid association until reconciliation confirms the transition. The protocol specification is intentionally minimal in this manuscript; a replication package should expose the concrete CRD schemas, CBOR field names, status enum values, timeout constants, and error codes used by the implementation.

6. Implementation

The implementation combines Rust control-plane services with Zephyr-based device firmware and WAMR execution. The API server, gateway, and controllers are implemented in Rust, using its memory-safety guarantees and asynchronous networking support. Resource interaction follows Kubernetes controller and status-patching practices [27, 46], allowing embedded endpoints to be managed as first-class resources without treating them as cluster nodes. On the device side, Zephyr provides the embedded networking and TLS substrate [47], while WAMR provides lightweight WASM execution on constrained targets [11].

The reference deployment runs on K3s [35]. In the presented framework, K3s denotes the concrete lightweight Kubernetes distribution used for the experiments, whereas K8s/Kubernetes denotes the generic API, manifests, CRDs, and controller model implemented by the framework and reflected in the repository naming. It uses namespace-scoped resources and locally built images, preserving real Kubernetes API and reconciliation semantics while remaining accessible on a single-node Linux environment for development, experimentation, and repeatable evaluation. The deployment includes the API services, dashboard, gateway/controller components, CRDs, and supporting services needed to exercise enrollment, deployment, and status propagation.

Responsibilities are distributed across specialized controllers rather than concentrated in a single service. Device-oriented reconciliation manages attachment state and gateway resolution. Application-oriented reconciliation interprets deployment intent and maps it to gateway actions. Gateway-oriented reconciliation maintains metadata needed for path resolution and status interpretation. This decomposition follows the CRD model and reduces the risk that unrelated concerns become coupled inside one control loop.

6.1. Control-Plane Components

The control-plane implementation is organized around a narrow API layer and a set of reconciliation workers. The API server is responsible for user-facing operations, validation of high-level requests, and creation or update of Kubernetes resources. It does not maintain authoritative device-session state. That responsibility is delegated to the gateway and reflected through CRD status fields, so that the Kubernetes API remains the persistent source of desired state and the gateway remains the source of southbound observations. This split prevents the dashboard or API server from becoming a hidden second control plane.

The Device controller watches Device resources and interprets gateway-reported attachment evidence. Its primary task is not to open device connections directly, but to maintain a consistent platform view: the selected gateway, the latest known connectivity phase, heartbeat freshness, and the last valid association between device identity and gateway session. The Application controller watches Application resources and turns a desired deployment into a gateway action only when the target device and mediation path can be resolved. The Gateway controller maintains gateway metadata and makes the mapping between cloud-side resources and fog-layer endpoints explicit. Table 5 summarizes these responsibilities.

Table 5
Implementation responsibilities across the validated stack.

Component	Main responsibility	State written or observed	Failure containment
Dashboard and API server	Accept operator requests, validate high-level parameters, and create or update CRD objects.	Application desired state, Device metadata, and Gateway references.	No direct southbound session ownership.
Device controller	Interpret attachment, phase, heartbeat freshness, and gateway association.	Device status fields and gateway-derived evidence.	Avoids replacing valid status from one transient missing observation.
Application controller	Resolve target device and gateway, dispatch deployment intent, and reconcile execution result.	Application status, deployment phase, and outcome evidence.	Keeps desired state intact if execution evidence is absent or inconsistent.
Gateway runtime	Terminate TLS sessions, decode CBOR frames, bind device identities, and forward deployment commands.	Runtime session map and protocol-derived status patches.	Isolates protocol errors from Kubernetes controllers and firmware.
Zephyr/WAMR firmware	Maintain the protected channel, emit liveness evidence, load the WASM module, and report execution result.	Heartbeat messages, deployment acknowledgments, and runtime result fields.	Does not write Kubernetes resources or store cluster credentials.

6.2. Gateway Runtime State

The gateway runtime maintains the small amount of mutable state that cannot naturally live inside Kubernetes alone: active TLS sessions, decoded device identifiers, freshness information for recently received heartbeats, and pending deployment operations. This state is intentionally treated as a cache of protocol reality rather than as a replacement for Kubernetes status. When a device enrolls, the gateway creates a session-to-device binding only after the enrollment material is compatible with the expected Device resource. When a heartbeat arrives, the gateway refreshes liveness evidence and exposes the new observation to the control plane. When an Application deployment is requested, the gateway checks that the target device is currently reachable through a known session before emitting the southbound command.

This organization gives the framework a precise failure boundary. If the firmware disconnects, the gateway loses the active transport session but the Device resource still retains the latest desired configuration and the latest valid reported association until reconciliation determines that the condition is stale. If the Kubernetes API is temporarily unavailable, the gateway can still preserve the immediate transport view, but durable convergence waits until status patches are accepted. If the gateway restarts, the cloud-side desired state survives and devices must re-establish session-bound evidence. These cases are not hidden behind a best-effort transport acknowledgment; they are reflected as missing, stale, or inconsistent evidence that controllers can interpret conservatively.

6.3. Embedded Execution Path

On the embedded side, the firmware implements only the southbound protocol roles required by the orchestration model. It establishes the protected channel, sends enrollment and heartbeat records, receives deployment commands, passes the WASM payload to WAMR, and returns a structured result. It does not embed Kubernetes client logic, service-account credentials, resource watches, or container-image management. The edge endpoint is therefore closer to a managed execution target than to a miniature cluster node. This is a deliberate implementation constraint: the more Kubernetes behavior is pushed into the firmware, the harder it becomes to keep the device portable, auditable, and suitable for constrained deployments.

The WASM payload used in the evaluation is intentionally minimal so that the measurement focuses on orchestration overhead rather than application computation. This choice separates two questions that are often conflated in edge experiments: whether a runtime can execute a non-trivial application efficiently, and whether an orchestration path can reliably deliver, start, and observe a workload. The present implementation addresses the second question. Larger WASM modules, richer WASI support, and application-specific input/output channels can be added above the same deployment and evidence path without changing the control ownership model.

7. Experimental Evaluation

7.1. Evaluation Design and Testbed

The evaluation combines (i) a functional validation campaign over the full enrollment-to-deployment path and (ii) a sustainability-oriented measurement pass on the same testbed. The functional study asks whether cloud intent, gateway mediation, and embedded WASM execution produce mutually consistent evidence under repeatable conditions. The sustainability pass quantifies southbound wire sizes, edge firmware footprint, and Kubernetes pod resource use. Neither pass claims measured joules or carbon reductions; they establish baselines relevant to green-experimentation follow-up.

Experiments run on a single-node K3s cluster with API server, gateway, application controller, and CRDs active. The southbound endpoint is a Renode-emulated STM32F746 running Zephyr firmware with WAMR and

Table 6
Experimental testbed and software configuration.

Item	Configuration
Host OS	Ubuntu 24.04 LTS (kernel 6.8.0), x86_64 KVM guest
Host resources	32 vCPUs, 62 GiB RAM, 293 GiB SSD
Orchestrator	K3s v1.35.4+k3s1, single control-plane node
Southbound emulation	Renode `antmicro/renode:nightly`, board `Stm32F746gDisco`
Firmware stack	Zephyr RTOS 3.5.0, WAMR 2.4.1, minimal WASM test module (33 B)
Device networking	Bridge `wasmbr0` at 192.168.1.1/24, one TAP per device, DHCP, DNAT to the gateway pod's TLS port

connects over TAP-based networking with DNAT to the gateway TLS port. Table 6 summarizes the host and software stack.

After one warm-up run, the campaign executes n=100 repeated enrollment-to-deployment trials started from a known logical state. Each trial performs enrollment verification, heartbeat supervision, and deployment of the minimal WASM module to the same emulated device. The campaign should therefore be interpreted as repeated logical validation of the same emulated device, gateway path, and control-plane workflow, rather than as evidence from independent physical devices or independent network environments. A trial is *transactionally* successful only if all three stages succeed and deployment evidence is consistent across the gateway runtime view and the Application CRD (`phase=Running`, device connected, fresh heartbeat). Continuous latencies use monotonic client-side timers; we report mean, standard deviation, median, p95, p99, IQR, CV, and 95% CI of the mean ($\bar{x} \pm t_{0.975,n-1}\, s/\sqrt{n}$, with $t_{0.975,99}$=1.984). These intervals summarize the repeated-trial campaign and are not intended to estimate variability across heterogeneous devices, gateways, or physical network conditions. For binary outcomes we use Wilson score intervals; with 100/100 successes the 95% lower bound is 96.3%.

Two caveats govern how these summaries should be read. First, consecutive trials share the same host, the same warmed page cache, the same already-compiled runtime and the same long-lived control-plane processes, so they are repetitions rather than independent draws, and any residual serial dependence they carry would distort an interval computed under an i.i.d. assumption. We therefore report the t-based intervals as descriptive dispersion summaries of this campaign rather than as inferential statements about a wider population of deployments, and we check the assumption directly on the trial series (Figure 6). The end-to-end series shows no accumulation and no drift: the lag-1 autocorrelation is −0.09, the Spearman rank correlation between latency and trial index is −0.14 (t=−1.42, not significant at n=100), and the first and second halves of the campaign have means of 1212.1 ms and 1159.5 ms with standard deviations of 184.7 ms and 142.2 ms, i.e. a difference well inside one standard deviation. The campaign is thus stationary after the discarded warm-up run, with no thermal or cache-warming regime and no positive correlation between neighbouring trials. Second, the campaign induces no faults: no packet loss, no gateway restart, no key mismatch, no contention from co-located tenants. Under those conditions a deterministic pipeline is expected *a priori* to succeed in every trial, so 100/100 confirms that the cross-layer evidence-convergence criterion of Table 7 holds whenever the underlying mechanism is exercised without induced failure; it does not estimate reliability under adverse conditions. The Wilson lower bound of 96.3% should accordingly be read as the arithmetic consequence of observing 100 successes out of 100 attempts under a fault-free protocol, and not as an availability figure for a production fleet, whose failure modes this campaign does not sample.

7.2. Trial Procedure and Evidence Semantics

Each trial is treated as a repeated logical execution of the enrollment-to-deployment workflow, not as evidence about independent physical devices or independent network environments. The harness starts from a known logical state, observes the device enrollment path, waits for heartbeat evidence to become visible through the gateway, creates or updates the Application intent, and then checks whether the gateway runtime view and Kubernetes status converge to the same deployment outcome. This definition is deliberately stricter than checking for a successful southbound message: a deployment command that reaches the device but is not reflected in Application status is counted as a failed transaction.

The measurement record separates four kinds of evidence. First, protocol evidence comes from enrollment, heartbeat, and deployment records exchanged over the TLS/CBOR channel. Second, gateway evidence comes from the runtime association between a device identifier and an active session. Third, Kubernetes evidence comes from Device and Application status fields after reconciliation. Fourth, timing evidence comes from the client-side harness using monotonic timestamps around the evaluated stages. A trial succeeds only when these evidence sources agree before the configured timeout. This convention reduces the risk of reporting success from a single eventually consistent field or from a transport acknowledgment that never becomes durable control-plane state.

Latency measurements follow the same separation. Enrollment latency measures the interval needed to complete the identity-binding path and expose the enrolled device to the control plane. Deployment latency measures the path from declared Application intent to reported execution outcome. End-to-end latency is the sum of enrollment and deployment for the evaluated transaction. Heartbeat observability is reported separately because it measures how quickly the harness can observe a fresh heartbeat after it is available through the gateway; it is not the firmware heartbeat period and should not be interpreted as a liveness interval.

Table 7
Trial-level success and failure interpretation.

Condition	Trial interpretation
Enrollment accepted but identity not matched to the expected Device resource	Failure, because a protected channel alone is not a platform-level attachment.
Heartbeat received by the gateway but not reflected as fresh liveness evidence	Failure for the transaction, because the control plane cannot rely on the observation.
Deploy command acknowledged but Application status does not converge	Failure, because command dispatch is not equivalent to verifiable execution.
Application status reports success while gateway or Device evidence is inconsistent	Failure, because cross-layer evidence does not agree.
Enrollment, heartbeat freshness, gateway association, and Application outcome agree	Transactional success.

7.3. Functional Validation Results

All 100 transactional trials completed successfully: enrollment, heartbeat supervision, deployment, and cross-layer evidence checks never failed. These trials exercise the enrollment path in which the Device resource reaches `phase = Connected` from gateway-observed session evidence, which is the path described in Section 5 and the one the framework uses in normal operation; Section 7.4.5 reports a separate, later instrumentation pass that provisions devices through the auxiliary HTTP registry endpoint instead, and the difference between the two is explained there. Because every measured stage succeeded in all trials, a per-stage success table would repeat identical Wilson bounds and is omitted; the meaningful variation lies in latency and its distribution.

Table 8 reports the latency profile. Mean end-to-end latency (enrollment plus deployment; heartbeat emission excluded and heartbeat observability reported separately) is 1286.0 ms with 95% CI [1226.4, 1345.6] ms. Deployment carries a heavy upper tail (CV 0.43, maximum 3000.8 ms against a median of 574.1 ms) that the mean and the p95 both understate; it reproduces across independent campaigns on this testbed and is the dominant contribution to end-to-end variability. The heartbeat row reflects *observability* latency, how quickly the measurement harness reads a fresh heartbeat from the gateway, not the 25 s firmware emission interval.

Figure 5 compares stage latency distributions on a logarithmic scale. Heartbeat observability is two orders of magnitude faster than enrollment or deployment. Figure 7 shows that deployment contributes most of the tail mass: its p95 (1031.3 ms) and p99 (1295.7 ms) exceed enrollment's, so

Table 8
Latency profile (n=100 trials). All values in milliseconds.

Stage	Mean	Median	p95	p99	IQR	CV	95% CI of mean
Enrollment	609.5	568.1	839.6	889.8	50.4	0.164	[589.6, 629.4]
Heartbeat obs.	4.08	4.01	4.97	5.29	0.53	0.114	[3.99, 4.17]
Deployment	676.5	574.1	1021.8	1145.2	228.4	0.428	[619.1, 733.9]
End-to-end	1286.0	1168.7	1661.5	1974.3	270.7	0.234	[1226.4, 1345.6]

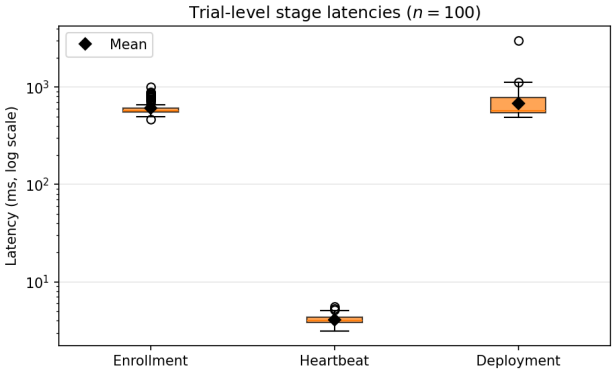

Figure 5: Distribution of trial-level stage latencies (n=100, log scale). Diamonds indicate sample means.

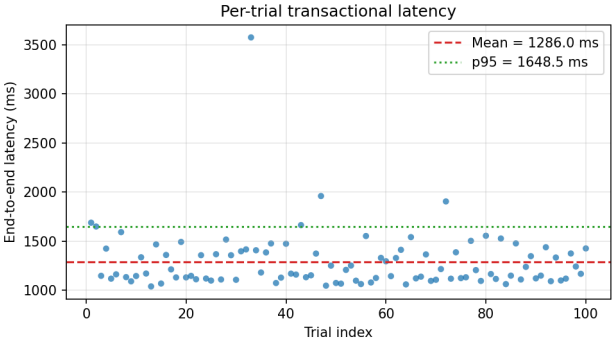

Figure 6: End-to-end latency by trial index (n=100). The series is stationary: lag-1 autocorrelation −0.09, Spearman ρ=−0.14 against trial index (not significant), half-campaign means 1212.1 ms and 1159.5 ms.

WASM orchestration, not TLS enrollment, dominates end-to-end variability (deployment CV = 0.288 vs. enrollment CV = 0.142).

7.4. Sustainability-Oriented Metrics

7.4.1. Southbound wire efficiency

Application messages use a 4-byte big-endian length prefix followed by CBOR payloads. Figure 8 summarizes measured wire sizes for representative messages. Heartbeat request and acknowledgment are 6 B each; a complete enrollment round trip totals 87 B of application payload; a minimal deploy round trip (33 B WASM module) totals 90 B. At the configured 25 s heartbeat period, steady-state application-layer heartbeat traffic is $12 \times (3600/25) = 1728$ B/h per device (≈1.69 KiB/h), excluding TLS record overhead

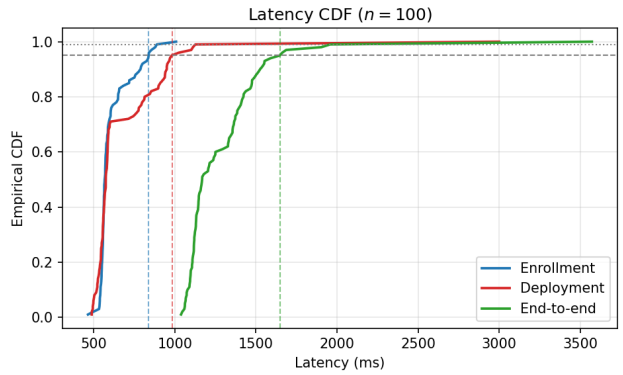

Figure 7: Empirical CDF of enrollment, deployment, and end-to-end latencies (n=100).

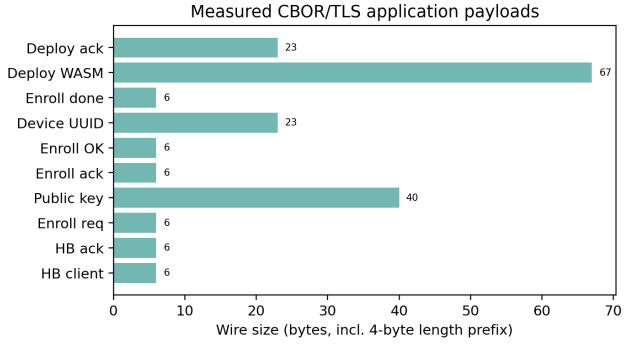

Figure 8: Application-layer wire sizes (4-byte length prefix + CBOR) for representative protocol messages.

and lower-layer TCP/IP, Ethernet/TAP, and retransmission overhead.

7.4.2. Edge footprint

The evaluated Zephyr/WAMR/TLS/CBOR firmware fits the STM32F746NG internal-memory budget: the flash-resident `zephyr.bin` image is 406,568 B (≈397 KiB), corresponding to text plus initialized data, and the static RAM allocation is 270,238 B (≈264 KiB), corresponding to data (9,384 B) plus BSS (260,854 B). The larger `zephyr.elf` file is 8,095,176 B (7.72 MiB) only because it includes ELF metadata and debug symbols for the Renode `sysbus LoadELF` workflow; it is not the image written to device flash. The benchmark WASM module remains 33 B. The architectural point supported by these measurements is therefore that embedded endpoints run no kubelet, container runtime, or Linux userspace while retaining an embedded-class memory footprint.

7.4.3. Concurrent devices

The trials above repeat one device through the full workflow, which measures the workflow but says nothing about concurrency. A separate pass therefore attaches three emulated STM32F746G devices at the same time. Each device carries its own enrollment key, its own link-layer identity and its own DHCP lease, so the gateway, the control plane and the network segment can all tell them apart; the identity a device presents during enrollment is the one recorded in its Device resource.

All three attach and stay attached. The gateway runtime view lists three distinct sessions, each with a heartbeat fresher than the 25 s firmware emission interval, and the three Device resources independently reach `phase = Connected` with their own liveness evidence. A single Application targeting all three devices converges to `Running` on each of them: the gateway dispatches three separate `DeployApplication` messages, one per session, and receives three successful deployment acknowledgments. Because a deployment command travels only over the session that belongs to its target device, three acknowledged deployments are direct evidence of three independent southbound channels rather than of one channel reused.

This establishes that concurrency is supported at the mechanism level, not that the design has been characterized at fleet scale. Three devices on a single emulation host exercise session multiplexing, per-device identity binding and per-device dispatch; they do not exercise gateway contention, multi-gateway placement, or the network conditions of a physical deployment.

7.4.4. Control-plane resources

Figure 9 reports mean Kubernetes pod usage (n=5 samples, one TLS-connected device, sampled while the trial campaign was running). The metric is the container working set reported by `metrics-server`, cross-checked against cgroup v2 `memory.current` and the anonymous resident set in `memory.stat`, which agree with it to within a few hundred kilobytes. The gateway averaged 3.0 millicores CPU and 4.4 MiB RAM (`memory.current` 5.60 MB, anonymous 3.82 MB), the API server averaged 169.2 millicores and 33.4 MiB under campaign load (`memory.current` 32.3–33.0 MB), and the application controller averaged 38.4 millicores and 12.0 MiB (`memory.current` 13.3 MB). Registering 50 synthetic boards through the gateway HTTP registry left gateway resident memory unchanged (`memory.current` identical before and after), indicating lightweight per-device mediation state at the fog layer.

7.4.5. Computational energy instrumentation (measured CPU-time; derived energy and carbon as illustrative proxies)

Beyond control-plane pod resources, we instrument a representative WASM computational workload directly. A Kubernetes pod running the fuel-metered Wasmtime runtime (`wasmbed-wasm-runtime`) repeatedly creates an isolated instance and executes a bounded arithmetic-loop WASM function (summing 0 to N), cycling 30 s idle and 30 s active phases. This instruments a control-plane/fog-layer WASM execution path as a computational-energy proxy; The pod runs under Guaranteed QoS (`requests = limits`, 2 vCPU) to remove self-induced CFS-throttling variance observed at the original 1-vCPU limit (835/27,021 accounting periods

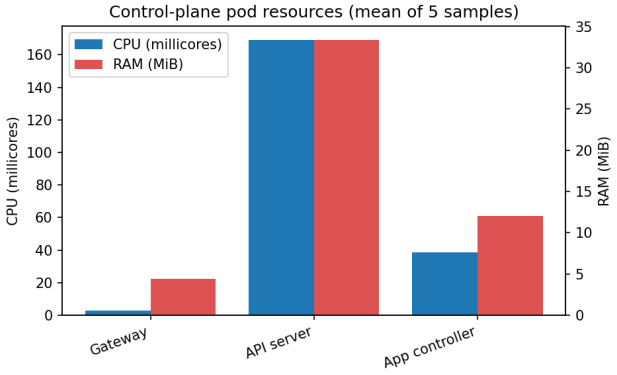

Figure 9: Mean CPU (millicores) and RAM (MiB) for control-plane pods during the sustainability measurement pass.

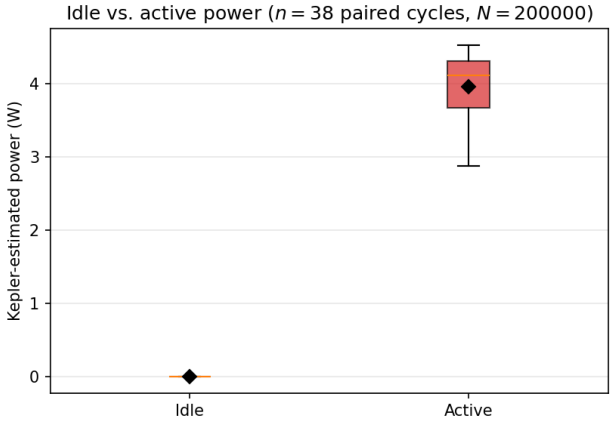

Figure 10: Kepler-estimated power, idle vs. active phase (n=38 paired 30 s cycles with a power reading, N=200,000). Model-based estimate (no RAPL on this host), not a hardware measurement; the estimator attributes exactly zero dynamic power to the idle windows.

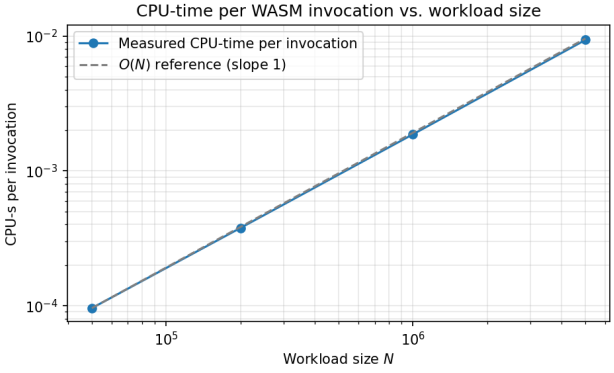

Figure 11: Measured CPU-time per WASM invocation vs. workload size N (log–log; cgroup `cpu.stat`, exact), against an $O(N)$ reference line of slope 1.

throttled under load, a scheduling artifact of the container's own quota, not of co-located tenants).

Two independent signals are collected. First, cgroup CPU time (`cpu.stat usage_usec`) is an exact kernel accounting figure, not an estimate: at N=200,000, across n=39 paired cycles, the pod consumed 29.9289 ± 0.0189 CPU-s during each 30 s active window versus 0.000116 ± 0.000053 CPU-s during each equal-length idle window (paired difference 95% CI [29.9230, 29.9354] s, excluding zero). Second, we deploy Kepler [45] as a DaemonSet and query `kepler_container_joules_total{mode="dynamic"}` scoped to the probe pod's own `pod_name` label, isolating it from co-located wasmbed pods in the same namespace. Because the host is a virtualized guest without exposed RAPL counters, Kepler's output is a regression-based power estimate rather than a direct hardware measurement; with that caveat, it shows a clear and physically plausible increase under load: it attributes no dynamic power to the pod during the idle windows and 3.952 W during the active ones (Figure 10). That the idle attribution is exactly zero rather than a small positive value is itself a property of the estimator, not a measurement of the pod drawing no power, and is a further reason to treat the wattage as secondary to the cgroup CPU-time. We treat the exact cgroup CPU-time as the primary, reproducible quantity throughout, and the Kepler wattage as a secondary, model-based signal consistent with it.

Repeating the same idle/active protocol while varying the workload size $N \in \{50{,}000, 200{,}000, 1{,}000{,}000, 5{,}000{,}000\}$ ($n \geq 10$ cycles per size) isolates CPU-time per WASM invocation from the fixed 30 s window: it grows from 95.9 μs at N=50,000 to 9.36 ms at N=5,000,000. The two largest sizes exceed the MCU profile's default Wasmtime fuel budget of 5×10^6 units, which traps the call; they are measured with the budget raised (fuel metering still enabled), a change that alters no other aspect of the workload. Plotted against an $O(N)$ reference line of slope 1 anchored at the smallest measured point (Figure 11), the four measurements track it closely across two orders of magnitude in N, consistent with the loop's linear time complexity rather than merely connecting four samples (Table 9; 95% CIs per point are narrower than the marker at this scale, $n \geq 10$ cycles each).

Converting CPU-seconds into Joules requires a hardware-specific power coefficient, and this is the one step of the pipeline that is not measured on this platform. We therefore state its provenance explicitly. The primary reference is the manufacturer specification for the deployed processor, the AMD EPYC 7313P: 16 cores / 32 threads with a default TDP of 155 W and a configurable TDP of 155–180 W [1], giving 4.84 W per vCPU at the default TDP. TDP is a thermal design limit rather than an instantaneous power draw, so it bounds rather than predicts consumption; as secondary corroboration, a third-party RAPL/`turbostat` measurement of the same single-socket part reports 35 W idle and 151 W full load across 32 threads [6], i.e. 1.09 W and 4.72 W per vCPU, within 3% of the specification-derived value. Neither figure is a measurement

Table 9
Workload size vs. measured CPU-time per invocation. CPU-s/call is exact kernel accounting; J/call is an *illustrative range*, never a measurement, spanning the two candidate per-vCPU full-load coefficients (4.72 W measured on the same part by a third party [6], 4.84 W from the manufacturer TDP [1]). The range does not bound the error of the linear power model itself (see text), so it is given to two significant digits and read as an order of magnitude.

N	n	CPU-s/call	J/call (illustrative)
50,000	10	9.59×10^{-5}	$4.5–4.6 \times 10^{-4}$
200,000	39	3.77×10^{-4}	1.8×10^{-3}
1,000,000	10	1.87×10^{-3}	$8.8–9.0 \times 10^{-3}$
5,000,000	10	9.36×10^{-3}	$4.4–4.5 \times 10^{-2}$

of this workload on this hardware, and a platform-specific coefficient would come from a documented energy benchmark such as SPECpower_ssj2008 [44] or the per-family coefficient table of the Cloud Carbon Footprint project [14]. The conversion also rests on a deliberately simple model: power proportional to allocated vCPU-time at full load, which ignores the idle baseline, DVFS and turbo behaviour, shared uncore, memory and I/O power attributable to no single vCPU, and the known non-linearity of power against utilization. Each effect can move the result by tens of percent in either direction, which is why every derived figure below is given to at most two significant digits. Removing the assumption requires anchoring the pipeline on a host exposing RAPL counters, so that at least one workload size yields a directly measured Joule figure; that run is the immediate next step and is not claimed here. Applying the Cloud Carbon Footprint average-wattage formula [14] to the measured utilization yields ≈119 J attributable to the active window above its idle baseline (≈1.5 mJ per WASM invocation over ≈79,300 invocations per window). Carbon follows from that energy and a grid emission factor, and is reported here parametrically for the same reason: at an intensity I, the active window accounts for $\approx 33\, I$ μgCO_2eq with I in gCO_2eq/kWh. We take I from the institutional source for Italy, the ISPRA national consumption-mix emission factor, 199.61 gCO_2eq/kWh for 2025 (preliminary estimate) [24], which puts the active window at ≈6.6 mgCO_2eq (of order 0.1 μgCO_2eq per invocation). Production- and consumption-mix factors differ and the choice must be stated with the value; the European Environment Agency indicator [19] provides the corresponding cross-country basis. The measured claims of this subsection are therefore the CPU-time quantities alone — the idle/active separation, the per-invocation CPU-time, and its linear growth with N; everything in Joules or grams of CO_2eq is a proxy computed from them with an external coefficient, evidence that the pipeline is measurement-ready rather than a validated energy or carbon claim, and no conclusion of this paper depends on its absolute value.

Per-phase, per-pod CPU-time (control-plane trials). As a minimal, additive instrumentation point layered on the trial harness behind Table 8 (no change to gateway, controller, or reconciler logic), each of the enrollment, heartbeat, and deployment stages is separately bracketed by a `cpu.stat` `usage_usec` snapshot for the gateway, API-server, and application-controller pods, giving CPU-seconds consumed by each pod during exactly that stage's wall-clock window. The measurement runs against the full live stack: a Zephyr 3.5.0/WAMR firmware image for the emulated STM32F746G target, connected to the gateway over TLS through the Renode TAP bridge. Table 10 reports a n=5-trial pass: all three stages succeed 5/5, with real, reproducible CPU-time per stage per pod, and the enrollment row covers the full protocol exchange (TLS handshake, CBOR EnrollmentRequest/PublicKey/DeviceUuid, gateway-side connection established) up to the Device CRD reaching `status.phase = Connected`.

Table 10
CPU-seconds per phase per pod (n=5 trials, provisioned through the auxiliary HTTP registry endpoint; all three stages succeeded 5/5). Exact cgroup `cpu.stat`, not an estimate.

Stage	Pod	Mean	Median	p95	95% CI of mean
Enrollment	gateway	0.000350	0.000326	0.000503	[0.000239, 0.000462]
Enrollment	api-server	0.143510	0.083265	0.384413	[−0.023697, 0.310717]
Enrollment	app-controller	0.036625	0.035557	0.045817	[0.029140, 0.044109]
Heartbeat	gateway	0.000413	0.000390	0.000684	[0.000206, 0.000620]
Heartbeat	api-server	0.020483	0.000000	0.102417	[−0.036379, 0.077345]
Heartbeat	app-controller	0.010088	0.010080	0.010956	[0.009043, 0.011133]
Deployment	gateway	0.006338	0.006521	0.008674	[0.003693, 0.008984]
Deployment	api-server	0.564172	0.465255	0.823915	[0.280249, 0.848096]
Deployment	app-controller	0.057557	0.027365	0.111654	[−0.002620, 0.117734]

Given n=5, this table demonstrates the instrumentation working end-to-end on real infrastructure rather than providing a statistically powered per-stage estimate; a n=100 run aligned with Table 8 is future work.

7.4.6. Architectural comparison

Table 11 contrasts gateway-mediated WASM orchestration with treating each constrained device as a Kubernetes worker. Edge Kubernetes deployments require a Linux-capable node with a kubelet, container runtime, and supporting operating-system services before user workloads, whereas the evaluated endpoint keeps those cluster agents off the device. Our design concentrates orchestration in a shared gateway (under 5 MiB measured) while keeping cluster agents off the device.

8. Discussion

8.1. Architectural and RI Implications

The main architectural lesson is that the gateway should be treated as a semantic boundary, not as a transparent relay. Constrained devices can remain focused on protected communication, local execution, and evidence emission, while the gateway handles identity validation, protocol translation, and status propagation toward Kubernetes. This separation avoids pushing cloud-facing semantics into firmware and gives the control plane a stable interface even if the southbound protocol evolves.

Table 11
Orchestration model comparison.

Aspect	Device as K8s worker	Gateway-mediated WASM edge
Edge runtime	kubelet + container runtime + OS	Zephyr + WAMR (394 KiB flash, 264 KiB RAM)
Edge cluster agent footprint	kubelet + container runtime + OS services required	none on device
Steady heartbeat traffic	CRI/container probes	1728 B/h per device at application layer, excluding TLS/lower layers
Fog mediation RAM	per-node agent	<5 MiB shared gateway

The evaluation confirms that protocol success alone is not sufficient: enrollment, liveness, and deployment evidence must be interpreted consistently before they become trustworthy control-plane state. This supports the central design decision to separate desired state, gateway-derived evidence, and controller reconciliation. Future extensions such as multi-gateway placement, staged rollout, or retry policies should preserve this separation rather than allowing multiple components to write conflicting interpretations of device status.

For sustainable digital research infrastructures, the same separation creates explicit attachment points for energy and carbon policies. Placement decisions can be expressed as desired state, gateway and device observations can be treated as evidence, and sustainability controllers can consume the same status model without requiring constrained devices to run additional cloud-native agents. The measured baselines in Section 7.4, byte-scale southbound traffic, single-digit-MiB gateway mediation, and no kubelet on the edge, suggest that subsequent green-experimentation studies can focus on policy and instrumentation rather than on redesigning the orchestration substrate.

For the scope of sustainable digital research infrastructures, the main outcome is therefore methodological rather than directly energetic: the paper identifies where orchestration evidence, device liveness, gateway mediation, and resource overhead can be captured consistently before energy-aware or carbon-aware decisions are automated.

The practical implication for research infrastructures is that orchestration evidence becomes a reusable experimental object. A placement policy, for example, can be evaluated by inspecting which Application resources were scheduled to which devices, which gateway mediated the execution, whether heartbeat evidence remained fresh, and whether the reported execution outcome matched the intended state. The same record can later be joined with external measurements such as site power, carbon-intensity annotations, thermal constraints, or gateway utilization. The current prototype does not optimize those signals, but it provides the control-plane hooks that a sustainability-aware controller would need in order to make such optimization auditable.

The gateway boundary also simplifies reproducibility. A reproduced experiment does not need the embedded device to expose Kubernetes-native behavior; it only needs the same southbound protocol semantics, the same CRD intent, and the same interpretation of evidence. This is useful for multi-site RIs, where physical boards, local networks, and energy meters may differ across laboratories. The orchestration claim can be retested with emulation, while hardware-specific claims can be added as separate measurement layers.

8.2. Validity and Scope

The validation targets system-level orchestration behavior rather than real-time performance, radio behavior, or sustainability optimization. Emulation and scripted trials are appropriate for exercising enrollment, deployment, status convergence, and reconciliation semantics, while physical boards, wireless links, and power meters require additional characterization. The results should therefore be interpreted as evidence that the orchestration model is coherent under the evaluated workflow.

External validity is bounded by the single-node K3s deployment, one gateway path, and a minimal WASM workload selected to isolate orchestration overhead rather than application-computation cost. Larger deployments introduce gateway contention, distributed failure modes, network partitions, and device mobility. Concurrency is exercised directly rather than assumed: three emulated devices attached simultaneously hold three distinct TLS sessions in the gateway runtime view and each receive their own deployment (Section 7.4.3). The 50-board registry exercise remains a registry-state scaling check, since those boards are registry entries rather than TLS-connected devices, and the concurrency evidence stops at the three devices actually measured on a single host: it does not license extrapolation to hundreds of devices, to multiple gateways, or to fleet-scale energy characterization, all of which remain open. Construct validity is also scoped: the study measures secure attachment, observable liveness, deployment completion, latency, wire sizes, firmware footprint, and control-plane RAM/CPU samples, while direct, hardware-validated energy consumption and long-duration reliability remain outside the current evaluation; Section 7.4.5 reports a computational-energy instrumentation pass on a representative WASM workload using exact cgroup CPU-time as the primary signal and a model-based Kepler host-power estimate as a secondary one, as a step toward that validation.

Scope of the energy signal. cgroup CPU-time (`cpu.stat`) is the only quantity in Section 7.4.5 that is an exact, reproducible measurement rather than a model output, and it alone should be treated as load-bearing for any energy claim in this paper. The Kepler wattage is a secondary, corroborating signal: this VM exposes no RAPL counters, so every reported watt is the output of Kepler's regression-based power

model rather than a hardware reading, independent of how large the sample is. The Joule and carbon figures in Table 9 are a further derived layer on top of the measured CPU-time, using a literature power coefficient, and are reported as illustrative estimates rather than measurements. A hardware-validated absolute wattage for this workload, on a host that combines a working k3s control plane with exposed RAPL counters, remains future work.

Conclusion validity is strengthened by the transactional success criterion: enrollment, heartbeat visibility, and deployment must succeed within the same trial. This avoids overstating correctness from isolated protocol stages. Broader claims about optimality, scalability, or comparative advantage require additional experiments against alternative orchestration designs and physical-device deployments.

8.3. Scalability and Failure Modes

The main scalability pressure point is the gateway, because it concentrates southbound sessions, protocol parsing, and status propagation. This is an intentional trade-off: moving the boundary into the gateway avoids embedding Kubernetes behavior in every device, but it also means that future deployments must treat gateways as schedulable and observable resources. Multi-gateway operation should therefore add explicit placement policies, per-gateway capacity metadata, and failover rules rather than relying on implicit network reachability. In such a design, a Device resource would declare or discover a preferred gateway, while Application reconciliation would resolve the current gateway association before attempting deployment.

The evaluated system also identifies four categories of faults. Transport faults include TLS interruption, device reboot, and heartbeat loss. Protocol faults include malformed CBOR records, identity mismatch, stale session state, and deployment results that cannot be associated with the requested Application. Control-plane faults include delayed status patches, controller restarts, and conflicting updates from independent reconcilers. Application faults include WASM load failure, runtime error, or an execution result that does not satisfy the expected policy. Treating these cases separately is important because each one requires a different recovery action: reconnect, reject enrollment, retry deployment, preserve desired state, or surface an explicit error to the operator.

The above-stated considerations also limits the interpretation of the present results. The 100-trial campaign shows that the nominal enrollment-to-deployment transaction is coherent under the evaluated emulated conditions. It does not establish maximum gateway capacity, behavior under network partitions, real-time scheduling guarantees, or recovery time after simultaneous failures. Those are natural next experiments once the single-device protocol and reconciliation invariants are fixed.

8.4. Reproducibility

The evaluation is designed to be repeatable on a commodity Linux host with Docker and K3s. Renode makes the embedded side deterministic enough for regression-style validation, although it remains complementary to future physical-board experiments. Evidence traceability is maintained across multiple layers: gateway logs, API-visible resource states, and Kubernetes CRD status describe the same workflow from different perspectives, so that protocol success and control-plane convergence are not inferred from a single eventually consistent field.

For a replication package, the most important artifacts are therefore not only the source code and container images, but also the CRD manifests, firmware configuration, WASM module, trial harness, raw latency records, and the scripts that generate the figures. Reporting these artifacts together would allow another researcher to distinguish implementation regressions from environmental variation. A failed replication could then be localized to a specific layer: Kubernetes resource creation, gateway mediation, TLS/CBOR communication, embedded execution, or post-processing of the measurement records. If institutional constraints prevent publishing all raw records, the artifact statement should separate public assets, such as source code, manifests, and scripts, from records available on request.

9. Conclusion and Future Work

This paper presented a Kubernetes-native framework for managing embedded WebAssembly workloads across the Cloud–Fog–Edge continuum without requiring constrained devices to become Kubernetes nodes. The contribution integrates CRD-based intent, gateway-mediated TLS/CBOR communication, Zephyr firmware, WAMR execution, and evidence-driven reconciliation into a single verifiable operational path suitable for sustainable digital research infrastructures.

The experimental evaluation on a K3s testbed with emulated embedded targets confirms that the architecture supports a complete enrollment-to-deployment workflow under repeatable conditions, with 100/100 transactional success and approximately 1.29 s mean end-to-end orchestration latency. The measured application-layer traffic, firmware footprint, and fog-layer resource use provide quantitative anchors for subsequent energy-aware placement, carbon-aware annotation, and green-experimentation studies, while direct energy optimization remains future work.

Future work will extend the platform toward multi-gateway deployments, fault injection, physical hardware, wattmeter- or RAPL-validated energy measurements, carbon-intensity annotation beyond the illustrative, derived-estimate pass reported in Section 7.4.5, and larger device populations.

Data Availability Statement

The data supporting the findings of this study consist of deployment logs, Kubernetes resource states, controller and gateway traces, and protocol evidence from controlled

experiments. The source repository provides the public deployment assets needed to inspect the framework. The measurement artifacts underlying every number reported in Section 7 — the per-trial latency and success records of the n=100 campaign, the paired idle/active and workload-size energy records, the per-phase `cpu.stat` records, the pod-resource samples, and the analysis and figure-generation scripts that turn them into the tables and figures of this paper — are available in the source repository below. Records not included in the repository are limited to raw operational logs that carry institutional infrastructure identifiers; these are available from the corresponding author upon reasonable request, subject to institutional policies and applicable confidentiality constraints.

Code Availability Statement

The source code and deployment assets associated with the framework are maintained in the public project repository `TODO-set-repository-url`. The exact revision evaluated in this paper is tagged `v1.0`, so that the results can be reproduced from a fixed snapshot rather than from a moving branch.

Funding

This research received no external funding.

Conflict of Interest

The authors declare that they have no conflict of interest related to this work.

Author Contributions

Conceptualization: Giovanni Merlino, with input from Francesco Longo. Methodology: Luca D'Agati, Giuseppe Tricomi, Fabio Orazio Mirto, and Giovanni Merlino. Software: Luca D'Agati, with contributions from Giuseppe Tricomi and Fabio Orazio Mirto. Validation: Luca D'Agati, Giuseppe Tricomi, and Fabio Orazio Mirto. Investigation and data curation: Luca D'Agati, Giuseppe Tricomi, and Fabio Orazio Mirto. Writing—original draft: Luca D'Agati, with writing support from Giuseppe Tricomi and Fabio Orazio Mirto. Writing—review and editing: Luca D'Agati, Giuseppe Tricomi and Fabio Orazio Mirto. Supervision: Giovanni Merlino, with support from Francesco Longo.

References

[1] Advanced Micro Devices, 2021. Amd epyc 7313p processor specifications. https://www.amd.com/en/products/processors/server/epyc/7003-series/amd-epyc-7313p.html. 16 cores / 32 threads; default TDP 155 W, configurable TDP 155–180 W; Accessed: 2026-08-19.

[2] Akri Authors, 2026. Akri: A kubernetes resource interface for the edge. https://docs.akri.sh/. Accessed: 2026-05-15.

[3] Amazon Web Services, 2026. Freertos documentation. https://www.freertos.org/. Accessed: 2026-05-15.

[4] Antmicro, 2026. Renode: Development framework for multi-node embedded networks. https://renode.io/. Accessed: 2026-05-15.

[5] Baccelli, E., Hahm, O., Gunes, M., Wahlisch, M., Schmidt, T.C., 2013. RIOT OS: Towards an os for the internet of things, in: Proceedings of the IEEE Conference on Computer Communications Workshops, pp. 79–80.

[6] Balci, M., 2023. Amd epyc 7313p energy consumption test. https://metebalci.com/blog/epyc-energy-consumption-test/. Third-party RAPL/turbostat PkgWatt measurement, single-socket, 32 threads, used here only as secondary empirical corroboration; Accessed: 2026-07-29.

[7] Bonomi, F., Milito, R., Zhu, J., Addepalli, S., 2012. Fog computing and its role in the internet of things, in: Proceedings of the First Edition of the MCC Workshop on Mobile Cloud Computing, pp. 13–16.

[8] Bormann, C., Hoffman, P., 2020. RFC 8949: Concise binary object representation (cbor). RFC 8949.

[9] Bytecode Alliance, 2026a. Wasmtime: A fast and secure runtime for webassembly. https://wasmtime.dev/. Accessed: 2026-05-15.

[10] Bytecode Alliance, 2026b. Webassembly system interface (wasi). https://wasi.dev/. Accessed: 2026-05-15.

[11] Bytecode Alliance and WAMR Contributors, 2026. Webassembly micro runtime (wamr). https://github.com/bytecodealliance/wasm-micro-runtime. Accessed: 2026-05-15.

[12] Canonical, 2026. Microk8s documentation. https://microk8s.io/docs. Accessed: 2026-05-15.

[13] Chiang, M., Zhang, T., 2016. Fog and iot: An overview of research opportunities. IEEE Internet of Things Journal 3, 854–864.

[14] Cloud Carbon Footprint, 2026. Cloud carbon footprint: Methodology. https://www.cloudcarbonfootprint.org/docs/methodology/. Accessed: 2026-07-29.

[15] containerd Contributors, 2026. runwasi: Webassembly runtime integration for containerd. https://github.com/containerd/runwasi. Accessed: 2026-05-15.

[16] D'Agati, L., Tricomi, G., Arena, M., Longo, F., Puliafito, A., Merlino, G., 2025. Iot orchestration in the compute continuum: Integrating kubernetes with stack4things, in: 2025 IEEE 25th International Symposium on Cluster, Cloud and Internet Computing Workshops (CCGridW), pp. 140–147. doi:10.1109/CCGridW65158.2025.00028.

[17] Dunkels, A., Gronvall, B., Voigt, T., 2004. Contiki – a lightweight and flexible operating system for tiny networked sensors, in: Proceedings of the 29th Annual IEEE International Conference on Local Computer Networks, pp. 455–462.

[18] EdgeX Foundry, 2026. Edgex foundry documentation. https://www.edgexfoundry.org/. Accessed: 2026-05-15.

[19] European Environment Agency, 2025. Greenhouse gas emission intensity of electricity generation in europe. https://www.eea.europa.eu/en/analysis/indicators/greenhouse-gas-emission-intensity-of-1. Country-level indicator; Accessed: 2026-08-19.

[20] Fdida, S., Makris, N., Korakis, T., Bruno, R., Passarella, A., Andreou, P., Belter, B., Crettaz, C., Dabbous, W., Demchenko, Y., et al., 2022. Slices, a scientific instrument for the networking community. Computer Communications 193, 189–203.

[21] Fermyon Technologies, 2026. Spin: Framework for webassembly applications. https://www.fermyon.com/spin. Accessed: 2026-05-15.

[22] FIWARE Foundation, 2026. Fiware documentation. https://www.fiware.org/. Accessed: 2026-05-15.

[23] Haas, A., Rossberg, A., et al., 2017. Bringing the web up to speed with webassembly, in: Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI), pp. 185–200.

[24] ISPRA – Istituto Superiore per la Protezione e la Ricerca Ambientale, 2026. Settore elettrico: emissioni di CO2 e altri impatti. Edizione 2026. Rapporti 430/2026. ISPRA. URL: https://www.isprambiente.gov.it/it/pubblicazioni/rapporti/settore-elettrico-emissioni-di-co2-e-altri-impatti-edizione-2026. national emission factors for electricity production and consumption.

[25] Krustlet Authors, 2026. Krustlet: Kubernetes kubelet in rust for webassembly workloads. https://krustlet.dev/. Accessed: 2026-05-15.

[26] KubeEdge Authors, 2026. Kubeedge documentation. https://kubeedge.io/. Accessed: 2026-05-15.
[27] Kubernetes Community, 2026. Operator pattern. https://kubernetes.io/docs/concepts/extend-kubernetes/operator/. Accessed: 2026-05-15.
[28] Levis, P., Madden, S., Polastre, J., Szewczyk, R., Whitehouse, K., Woo, A., Gay, D., Hill, J., Welsh, M., Brewer, E., Culler, D., 2005. Tinyos: An operating system for sensor networks. Ambient Intelligence , 115–148.
[29] OASIS, 2019. MQTT version 5.0. OASIS Standard.
[30] oneM2M, 2026. onem2m technical specifications. https://www.onem2m.org/technical/published-specifications. Accessed: 2026-05-15.
[31] OPC Foundation, 2026. Opc unified architecture specification. https://opcfoundation.org/about/opc-technologies/opc-ua/. Accessed: 2026-05-15.
[32] Open Mobile Alliance, 2022. Lightweight machine to machine technical specification. OMA SpecWorks.
[33] OpenYurt Authors, 2026. Openyurt: Extending kubernetes to edge computing. https://openyurt.io/. Accessed: 2026-05-15.
[34] Osterlind, F., Dunkels, A., Eriksson, J., Finne, N., Voigt, T., 2006. Cross-level sensor network simulation with cooja, in: Proceedings of the 31st IEEE Conference on Local Computer Networks, pp. 641–648.
[35] Rancher Labs, 2026. K3s: Lightweight kubernetes. https://k3s.io/. Accessed: 2026-05-15.
[36] Rescorla, E., 2018. RFC 8446: The transport layer security (tls) protocol version 1.3. RFC 8446.
[37] Rescorla, E., Tschofenig, H., Modadugu, N., 2022. RFC 9147: The datagram transport layer security (dtls) protocol version 1.3. RFC 9147.
[38] Riley, G.F., Henderson, T.R., 2010. The ns-3 network simulator, in: Modeling and Tools for Network Simulation, Springer. pp. 15–34.
[39] Satyanarayanan, M., 2017. The emergence of edge computing. Computer 50, 30–39.
[40] Schaad, J., 2022. RFC 9052: Cbor object signing and encryption (cose): Structures and process. RFC 9052.
[41] Selander, G., Mattsson, J., Palombini, F., Seitz, L., 2019. RFC 8613: Object security for constrained restful environments (oscore). RFC 8613.
[42] Shelby, Z., Hartke, K., Bormann, C., 2014. RFC 7252: The constrained application protocol (coap). RFC 7252.
[43] Shi, W., Cao, J., Zhang, Q., Li, Y., Xu, L., 2016. Edge computing: Vision and challenges. IEEE Internet of Things Journal 3, 637–646.
[44] Standard Performance Evaluation Corporation, 2008. SPECpower_ssj2008. https://www.spec.org/power_ssj2008/. Server power and performance benchmark; per-platform published results; Accessed: 2026-08-19.
[45] Sustainable Computing IO, 2026. Kepler: Kubernetes-based efficient power level exporter. https://sustainable-computing.io/. Accessed: 2026-07-29.
[46] The Kubernetes Authors, 2026. Kubernetes documentation. https://kubernetes.io/docs/. Accessed: 2026-05-15.
[47] The Zephyr Project, 2026. Zephyr project documentation. https://docs.zephyrproject.org/. Accessed: 2026-05-15.
[48] Varga, A., Hornig, R., 2008. An overview of the omnet++ simulation environment. Proceedings of the 1st International Conference on Simulation Tools and Techniques for Communications, Networks and Systems .
[49] Wasm3 Authors, 2026. Wasm3: A fast webassembly interpreter. https://github.com/wasm3/wasm3. Accessed: 2026-05-15.
[50] WasmEdge Authors, 2026. Wasmedge runtime. https://wasmedge.org/. Accessed: 2026-05-15.
[51] World Wide Web Consortium, 2019. Webassembly core specification. W3C Recommendation.